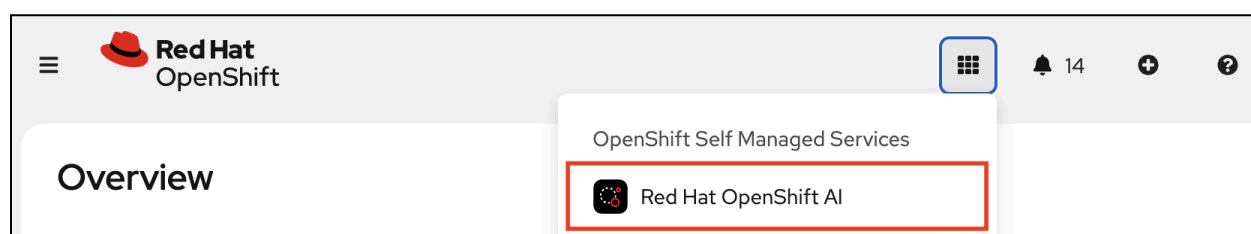


# Navigating to the OpenShift AI dashboard

You can access the OpenShift AI dashboard from the OpenShift console. From the dashboard, you can verify that the installed OpenShift AI version matches the version used in this tutorial.

## Procedure

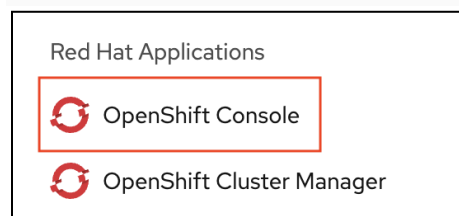
1. After you log in to the OpenShift console, click the application launcher icon on the header.



2. Log in to the OpenShift AI dashboard by using your OpenShift credentials. OpenShift AI uses the same credentials as OpenShift for the dashboard, notebooks, and all other components.

The OpenShift AI dashboard shows the Home page.

**Note:** You can navigate back to the OpenShift console by clicking the application launcher.



For now, stay in the OpenShift AI dashboard.

3. Make sure that the version of this tutorial matches the OpenShift AI version on your cluster.

Red Hat OpenShift AI v3.4 is the version described in this tutorial.

To find the OpenShift AI version for your cluster:

- a. In the top navigation bar of the OpenShift AI dashboard, next to your username, click the help icon () and then select **About**.

The **About** page shows the installed version.

4. Check the [Red Hat OpenShift AI Supported Configurations for 3.x](#) page to verify that your installed version is a supported version.
5. If this version of the tutorial does not match the installed version, access the matching version of the tutorial:
  - a. Navigate to the [Red Hat OpenShift AI documentation page](#).
  - b. Select the matching version from the drop-down list.

**Next step** Setting up your project

# Setting up your project

To implement a data science workflow, you must create a project as described in the following procedure. Projects help your team to organize and work together on resources within separated namespaces. From a project, you can create many workbenches, each with its own IDE environment (for example, JupyterLab), and each with its own connections and cluster storage. In addition, the workbenches can share models and data with pipelines and model servers.

## Prerequisites

- You have logged in to **Red Hat OpenShift AI**.

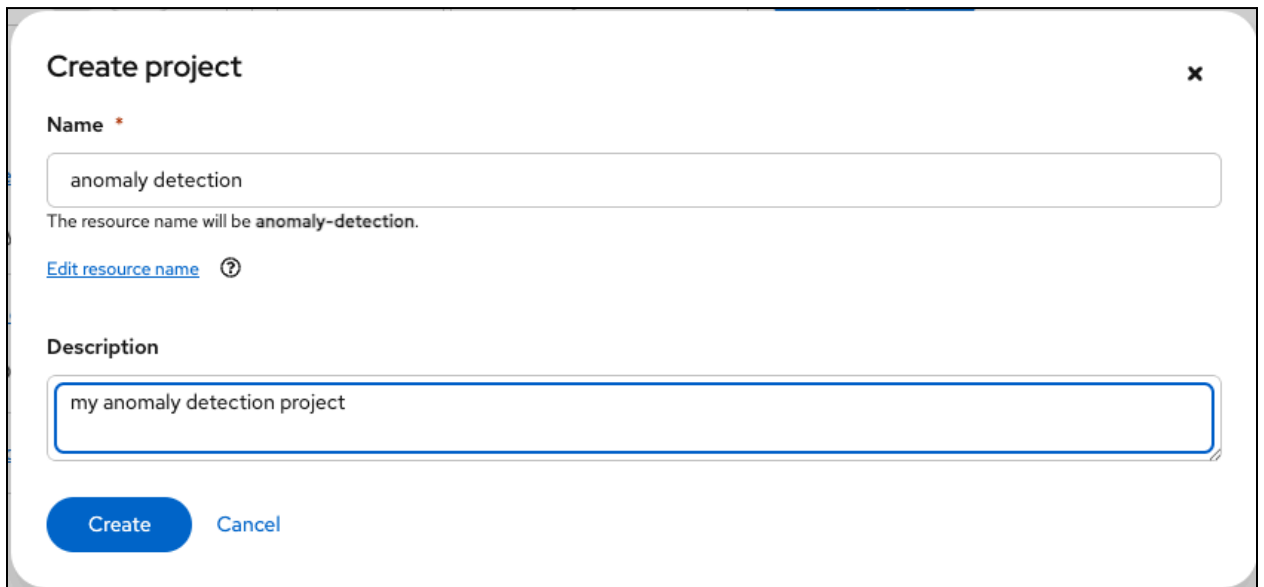
## Procedure

1. From the left navigation menu, select **Projects**. This page lists any existing projects that you have access to. You can select an existing project (if any) or create a new one.

2. Click **Create project**.

**Note:** You can start a Jupyter notebook by clicking the Start basic workbench button. However, in that case, it is a one-off Jupyter notebook run in isolation.

3. In the **Create project** modal, enter a display name and description.



**Create project** ✕

**Name \***

anomaly detection

The resource name will be anomaly-detection.

[Edit resource name](#) ?

**Description**

my anomaly detection project

**Create** **Cancel**

4. Click **Create**.

## Verification

Your project opens in the dashboard.

You can click the tabs to view more information about the project components and project access permissions:

- **Workbenches** are instances of your development and experimentation environment. They typically contain individual development environments (IDEs), such as JupyterLab, RStudio, and Code Server.
- **Pipelines** are a structured series of processes that collect, process, analyze, and visualize data. With AI pipelines, you can automate the execution of notebooks and Python code. By using pipelines, you can run long training jobs or retrain your models on a schedule without having to manually run them in a notebook.
- **Deployments** for quickly serving a trained model. A model server is a container image for a machine learning model. It exposes APIs to receive data, run the data through a trained model, and delivers a result (for example, a fraud alert).
- **Cluster storage** is a persistent volume that retains the files and data you're working on within a workbench. A workbench has access to one or more cluster storage instances.
- **Connections** contain object data which you can use for purposes such as configuration parameters and storing models, data, or artifacts.
- **Permissions** define which users and groups can access the project.

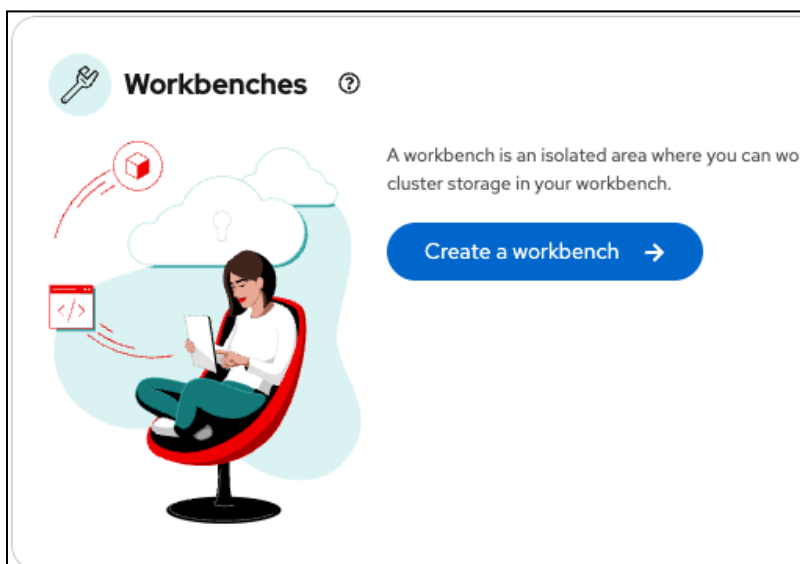
**Next step** Creating a workbench

# Creating a workbench

A workbench is an instance of your development and experimentation environment. When you create a workbench, you select a workbench image that has the tools and libraries that you need for developing models.

## Procedure

1. Navigate to the project detail page for the project that you created in **Setting up your project**.
2. Click the **Workbenches** tab, and then click the **Create workbench** button.



1. Enter the name and description.

**Name \***

anomaly detection

The resource name will be `anomaly-detection`.

[Edit resource name](#) ?

**Description**

my anomaly detection workbench

2. Red Hat provides several supported workbench images. In the Workbench image section, you can select one of the default images or a custom image that an administrator has set up for you. The Tensorflow image has the libraries needed for this workshop.
- 3.
4. For the workbench image:
  - a. Select the image that includes Tensorflow:  
Jupyter | Data Science | CPU | Python 3.12
  - b. For the image version, select 2025.2.

**Workbench image**

**Image selection \***

Jupyter | Data Science | CPU | Python 3.12

Python v3.12

**Version selection \***

3.4

3.4 (d3137ca)  
Software: Python v3.12

2025.2 (a2a3d89)  
Software: Python v3.12

Latest

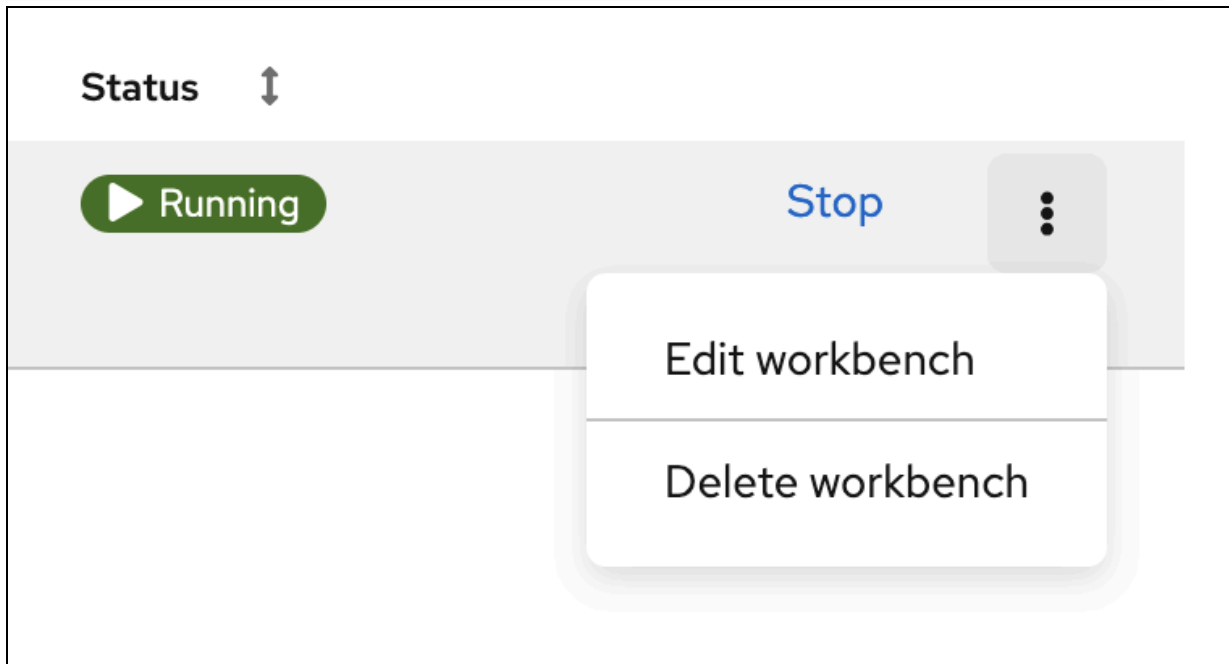
5. Under the Deployment size section, for Hardware profile, select `default-profile`.
6. Keep the default environment variables and storage options.
7. Click Create workbench.

## Verification

In the Workbenches tab for the project, the status of the workbench changes from `Starting` to `Running`.

| Overview Workbenches Deployments Cluster storage Connections Permissions |                                                                |                                  |          |                      |   |
|--------------------------------------------------------------------------|----------------------------------------------------------------|----------------------------------|----------|----------------------|---|
| Workbenches ⓘ                                                            |                                                                | <a href="#">Create workbench</a> |          |                      |   |
| Name ↑                                                                   | Workbench image                                                | Hardware profile                 | Status ↑ |                      |   |
| > <a href="#">anomaly detection</a> ⓘ<br>my anomaly detection workbench  | Jupyter   Data Science   CPU   Python 3.12<br>2025.2 (a2a3d89) | default-profile                  | Ready    | <a href="#">Stop</a> | ⋮ |

Note: If you made a mistake, you can edit the workbench to make changes.



### Next step

Importing the workshop files into the JupyterLab environment

# Importing the workshop files into the JupyterLab environment

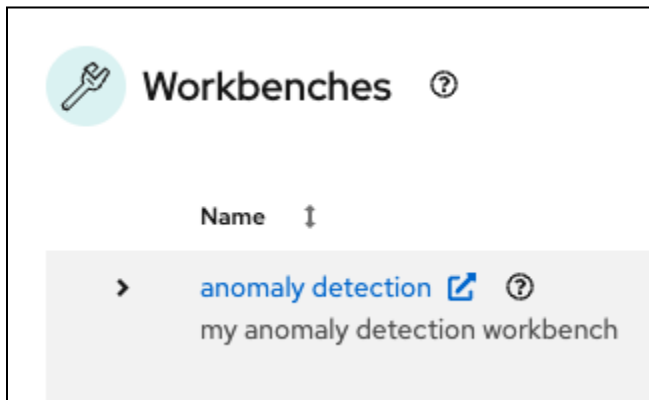
The JupyterLab environment is web-based, but all operations run on Red Hat OpenShift AI and are backed by the OpenShift cluster. This configuration allows you to run notebooks without installing local tools or consuming local CPU, GPU, or memory resources.

## Prerequisites

You created a workbench, as described in [Creating a workbench and selecting a workbench image](#).

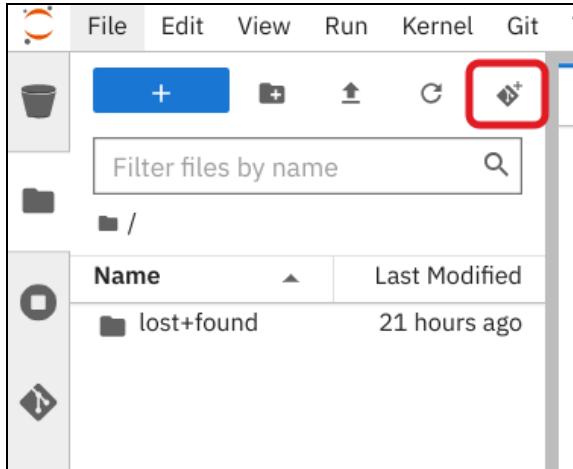
## Procedure

1. In the Workbenches tab for your project, click the link for your workbench. If prompted, log in and allow JupyterLab to authorize your user.

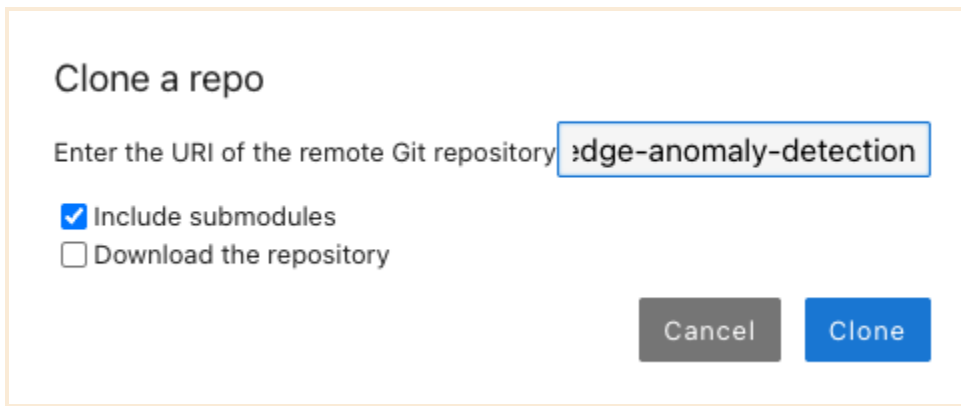


2. Your JupyterLab environment window opens.  
This file-browser window shows the files and folders that are saved inside your own personal space in OpenShift AI.
3. Bring the content of this workshop inside your JupyterLab environment:
  - a. On the toolbar, click the Git Clone icon:

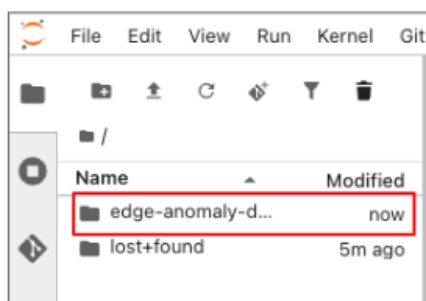




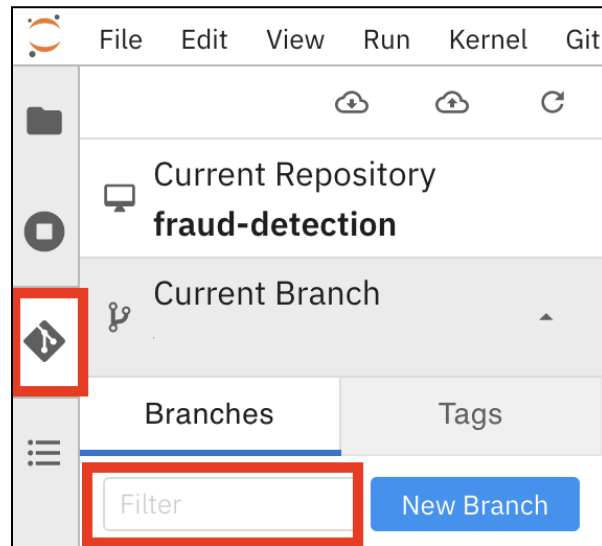
- a. Enter the following workshop Git https URL:
- b. `https://github.com/adrezni/edge-anomaly-detection`



- c. Select the Include submodules option, and then click Clone.
- d. In the file browser, double-click the newly-created fraud-detection folder to expand its contents.



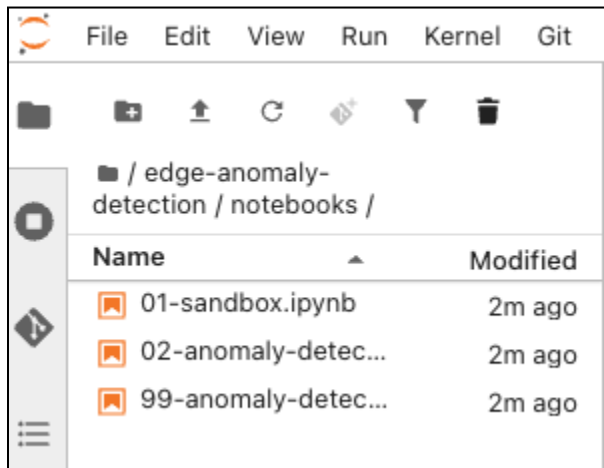
- e. In the left navigation bar, click the Git icon, and then click Current Branch to expand the branches and tags selector panel.
  - i. On the Branches tab, in the Filter field, enter main.



- f. Select origin/main.  
The current branch changes to main.

## Verification

1. In the left navigation bar, click the file browser icon to view the notebooks that you cloned from Git.



2. Verify that the Git version at the bottom of the JupyterLab window is main.

Next step

## Running code in a notebook

# Running code in a notebook

**Note:** If you're already at ease with Jupyter, you can skip to the next section.

A notebook is an environment where you have *cells* that can display formatted text or code.

This is an empty cell:



This is a cell with some code:



Code cells contain Python code that you can run interactively. You can edit the code and then run it. The code does not run on your computer or in the browser, but directly in your connected environment, Red Hat OpenShift AI in your case.

You can run a code cell from the notebook interface or from the keyboard:

- From the user interface: Select the cell (by clicking inside the cell or to the left side of the cell) and then click Run from the toolbar.



- From the keyboard: Press **CTRL + ENTER** to run a cell or press **SHIFT + ENTER** to run the cell and automatically select the next one.

After you run a cell, you can see the result of its code and information about when the code in the cell ran, as shown in this example:

```
In [2]: def print_some_text(entered_text):
        print('This is what you entered:' + entered_text)

        my_text = 'Hello world!'
        print_some_text(my_text)
```

executed in 9ms, finished 14:47:30 2021-04-13

This is what you entered:Hello world!

When you save a notebook, the code and the results are saved. You can reopen the notebook to view the results without having to run the program again, and still having access to the code.

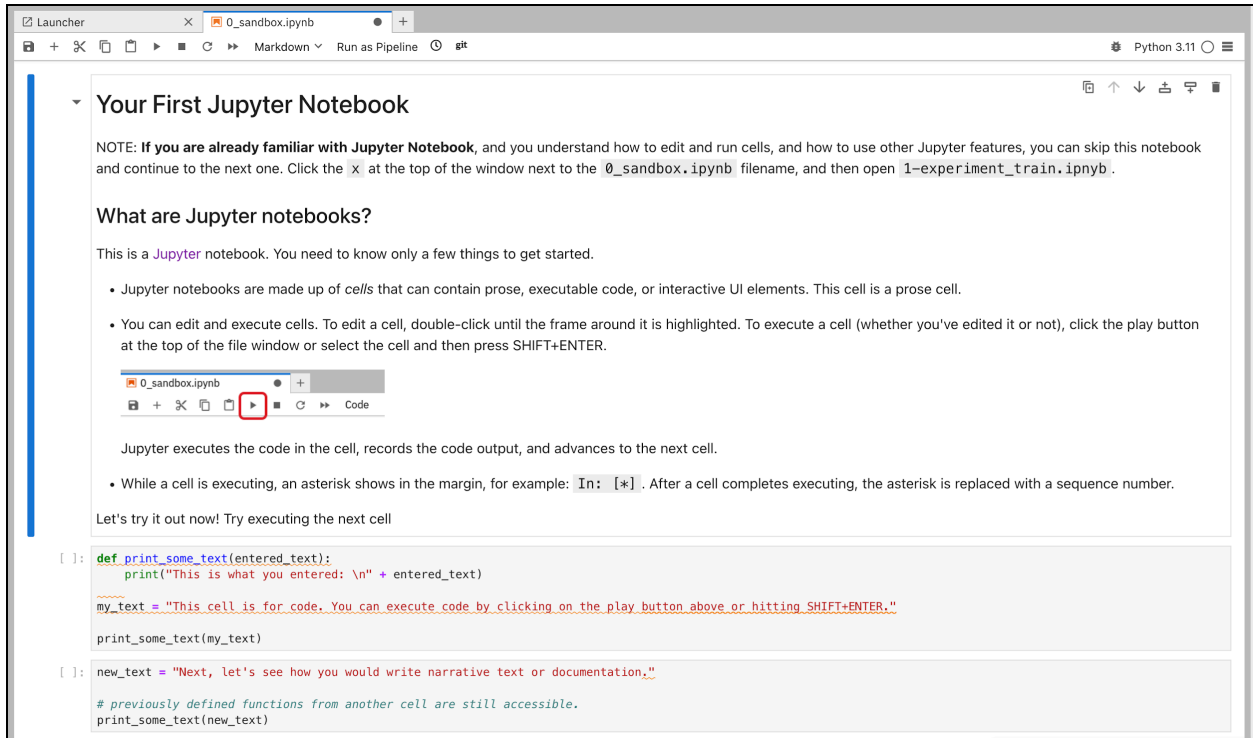
Notebooks are so named because they are like a physical *notebook*: you can take notes about your experiments, along with the code itself, including any parameters that you set. You can see the output of the experiment inline (this is the result after a cell runs), along with all the notes that you want to take. To take notes, from the menu switch the cell type from `Code` to `Markdown`.

### Prerequisites

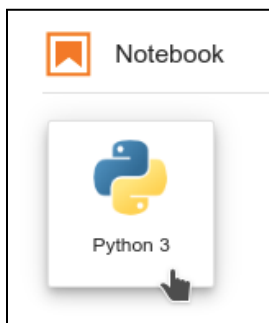
- You have imported the workshop files into your JupyterLab environment as described in [Importing the workshop files into the JupyterLab environment](#).

### Procedure

1. In your JupyterLab environment, locate the `0_sandbox.ipynb` file and double-click it to launch the notebook. The notebook opens in a new tab in the content section of the environment.



2. Experiment by, for example, running the existing cells, adding more cells and creating functions.
3. You can do what you want - it is your environment and there is no risk of breaking anything or impacting other users. This environment isolation is also a great advantage brought by OpenShift AI.
4. Optionally, create a new notebook in which the code cells are run by using a Python 3 kernel:
  - a. Create a new notebook by either selecting File → New → Notebook or by clicking the Python 3 tile in the Notebook section of the launcher window:



- b. You can use different kernels, with different languages or versions, to run in your notebook.

Next step **Explore Anomaly Detection**

Run cells in the 02-anomaly-detection.ipynb notebook